

SPRINGDALE SENIOR SECONDARY SCHOOL, JORHAT



SESSION 2025-26

CLASS & SECTION: XII
Science

SUBJECT: Computer
Science

SUBMITTED BY

STUDENT'S NAME: Jyotishman Chetia

ROLL NO: 12

GUIDED BY

TEACHER'S NAME: Ms. Teacher Name

DEGIGNATION: PGT Computer Science

CERTIFICATE

This is to certify that the Project/Practical/Internal Assessment report on "Fitness & Gym Tracker" is submitted by Jyotishman Chetia of Class XII Science .

The project is a part of practical/Internal Assessment of Annual Examination 2025-26 in the subject **Computer Science** conducted by Springdale High School, Jorhat (Assam).

This project is evaluated on

NAME

SIGNATURE

INTERNAL EXAMINER:

EXTERNAL EXAMINER:

HEAD OF INSTITUTION:

ACKNOWLEDGEMENT

I express my sincere gratitude towards my guide Mr./Ms./Mrs. **Ms. Teacher**
Name for his extended guidance and support during the preparation of this
project work.

I extend my special thanks to Mr./Ms./Mrs. _____
Principal, Springdale Senior Secondary School, Jorhat for his/her great support
by motivating and encouraging in every endeavour of ours.

A special thanks to CBSE for introducing the scheme of Project Work/
Practical/ Internal Assessment to make students discover their inbuilt abilities.

Last but not the least, I would like to thank my friends and family for their
cooperation and support.

Signature: _____

Student's Name: Jyotishman Chetia

Class: XII Science

Roll No: 12

School: Springdale High School, Jorhat

INDEX

S.No	Topic	Page No.
1	Cover Page	1
2	Project Certificate	2
3	Acknowledgment	3
4	Index	4
5	Overview of Python & MySQL	5
6	Need for the Project / Synopsis	6
7	Requirements & User Manual	7
8	Database Tables & Description	8
9	Module/Library Files Used	9
10	Source Code	10-16
11	Output Screens	17-18
12	Limitations	19
13	References	20
14	Thank you	21

OVERVIEW OF PYTHON & MYSQL

Python

Python is a high-level, object-oriented programming language known for its clean syntax and readability. It supports procedural, functional, and object-oriented programming paradigms. Python is widely used in software development, machine learning, automation, and database applications.

mysql-connector-python

Python uses external libraries such as `mysql-connector-python` to connect Python programs with MySQL databases. This library supports:

- ❖ Connecting to local/remote databases
- ❖ Executing SQL commands
- ❖ Performing CRUD operations (Create, Read, Update, Delete)

MySQL

MySQL is an open-source relational database management system (RDBMS). It stores data in the form of tables and supports:

- ❖ Structured Query Language (SQL)
- ❖ Primary & foreign keys
- ❖ Joins, constraints & transactions
- ❖ High-speed data processing

NEED FOR THE PROJECT / SYNOPSIS

Fitness centers and gyms often struggle with managing member records, trainer assignments, and specific workout plans using manual registers. Manual systems can lead to:

- ❖ Lost or disorganized data
- ❖ Inefficient tracking of member plans
- ❖ Difficulty in searching for specific member details

To solve these issues, this project "**Fitness & Gym Tracker**" automates the entire gym management process.

The system allows administrators to:

- ❖ Manage trainers and their specializations
- ❖ Register new members securely
- ❖ Assign specific 6-day workout plans to members
- ❖ Instantly search, view, or delete records

This project provides a simple, secure, menu-driven gym system suitable for small to medium-sized fitness centers.

REQUIREMENTS & USER MANUAL

Hardware Requirements

- ❖ A computer with minimum 4 GB RAM
- ❖ Intel/AMD Processor
- ❖ 500 MB free storage
- ❖ Keyboard for input

Software Requirements

- ❖ Python 3.x
- ❖ MySQL Server
- ❖ mysql-connector-python library
- ❖ Command Prompt / Terminal

How to Run the Project

1. Open Command Prompt
2. Navigate to the Python file location
3. Run the program using: `python gym_tracker.py`
4. The program automatically creates the database & tables if they don't exist.
5. Use the menu-driven interface to register members, add trainers, and assign plans.

DATABASE TABLES & DESCRIPTION

Database Name: gym_tracker_db

1. Table: trainers

This table stores information about every personal trainer.

- ❖ **trainer_id:** An integer that works as the primary key (Auto Increment).
- ❖ **name:** A text field (VARCHAR 100) to store the trainer's full name.
- ❖ **specialization:** A text field (VARCHAR 100) storing their expertise.
- ❖ **phone:** A text field (VARCHAR 15) marked as UNIQUE to avoid duplicates.

2. Table: members

This table stores the details of each registered member.

- ❖ **member_id:** An auto-incrementing integer (Primary Key).
- ❖ **name:** A text field (VARCHAR 100) storing the member's name.
- ❖ **age:** An integer field for age.
- ❖ **phone:** A text field (VARCHAR 15) marked as UNIQUE.
- ❖ **join_date:** A DATE field storing the registration date.
- ❖ **trainer_id:** A foreign key linking to the trainers table.

3. Table: workout_plans

This table stores the detailed 6-day routines assigned to members.

LIBRARY / MODULE FILES USED

1. mysql.connector

This is the main library used in the project.

It allows Python to connect to the MySQL database, create tables, insert data, update records, delete records, and run SQL queries.

It is also used for handling database errors using exception blocks.

2. sys

The sys module is used to safely exit the program in case a major error occurs, such as failure to connect to the MySQL database.

3. os

The os module is used to clear the terminal screen (`cls` or `clear`) to create a clean, refreshing menu-driven interface.

4. Built-in Python Features

The project also uses several built-in Python features such as:

- ❖ `input()` for taking user input
- ❖ `print()` for displaying menus and messages
- ❖ `try-except` for handling errors

❖ `match-case` for modern menu routing

SOURCE CODE

```
import mysql.connector

con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="Cbse@2026"
)

cur = con.cursor()

cur.execute("CREATE DATABASE IF NOT EXISTS gym_management")
cur.execute("USE gym_management")

cur.execute("""
CREATE TABLE IF NOT EXISTS trainers (
    trainer_id INT AUTO_INCREMENT PRIMARY KEY,
    trainer_name VARCHAR(50) NOT NULL,
    specialization VARCHAR(50)
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS members (
    member_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT,
    phone VARCHAR(15) UNIQUE,
    trainer_id INT
)
""")

def read_text(message, maximum):
    while True:
        text = input(message).strip()

        if text != "" and len(text) <= maximum:
            return text

        print("Enter between 1 and", maximum, "characters.")

def read_phone():
    while True:
        phone = input("Enter phone number: ").strip()
```

```

        if phone.isdigit() and len(phone) == 10:
            return phone

    print("Phone number must contain exactly 10 digits.")

def add_member():
    name = read_text("Enter name: ", 50)

    age = input("Enter age (10-100): ").strip()

    if not age.isdigit():
        print("Age must be a number.")
        return

    age = int(age)

    if age < 10 or age > 100:
        print("Age must be between 10 and 100.")
        return

    phone = read_phone()

    cur.execute(
        "SELECT member_id FROM members WHERE phone = %s",
        (phone,)
    )

    if cur.fetchone() is not None:
        print("Phone number is already registered.")
        return

    trainer_id = input("Enter trainer ID (optional): ").strip()

    if trainer_id == "":
        trainer_id = None
    else:
        if not trainer_id.isdigit():
            print("Trainer ID must be a number.")
            return

        cur.execute(
            "SELECT trainer_id FROM trainers WHERE trainer_id = %s",
            (trainer_id,)
        )

```

```

    )

    if cur.fetchone() is None:
        print("Trainer not found.")
        return

    cur.execute("""
        INSERT INTO members (name, age, phone, trainer_id)
        VALUES (%s, %s, %s, %s)
        """, (name, age, phone, trainer_id))

    con.commit()
    print("Member added! Member ID:", cur.lastrowid)

def display_member(row):
    print("\n-----")
    print("Member ID:", row[0])
    print("Name:", row[1])
    print("Age:", row[2])
    print("Phone:", row[3])

    if row[4] is None:
        print("Trainer: Not assigned")
    else:
        print("Trainer ID:", row[4])

def view_members():
    cur.execute("""
        SELECT member_id, name, age, phone, trainer_id
        FROM members
        """)

    rows = cur.fetchall()

    if len(rows) == 0:
        print("No members found.")
    else:
        for row in rows:
            display_member(row)

def search_member():
    member_id = input("Enter member ID: ").strip()

```

```

if not member_id.isdigit():
    print("Enter a numeric ID.")
    return

cur.execute("""
    SELECT member_id, name, age, phone, trainer_id
    FROM members
    WHERE member_id = %s
""", (member_id,))

row = cur.fetchone()

if row is None:
    print("Member not found.")
else:
    display_member(row)

def update_member():
    member_id = input("Enter member ID: ").strip()

    if not member_id.isdigit():
        print("Enter a numeric ID.")
        return

    cur.execute(
        "SELECT member_id FROM members WHERE member_id = %s",
        (member_id,)
    )

    if cur.fetchone() is None:
        print("Member not found.")
        return

    phone = read_phone()

    cur.execute("""
        SELECT member_id FROM members
        WHERE phone = %s AND member_id != %s
""", (phone, member_id))

    if cur.fetchone() is not None:
        print("Another member already uses this phone number.")
        return

```

```

    cur.execute(
        "UPDATE members SET phone = %s WHERE member_id = %s",
        (phone, member_id)
    )

    con.commit()
    print("Phone number updated!")

def delete_member():
    member_id = input("Enter member ID: ").strip()

    if not member_id.isdigit():
        print("Enter a numeric ID.")
        return

    confirm = input("Delete this member? (y/n): ").strip().lower()

    if confirm != "y":
        print("Deletion cancelled.")
        return

    cur.execute(
        "DELETE FROM members WHERE member_id = %s",
        (member_id,)
    )

    con.commit()

    if cur.rowcount == 0:
        print("Member not found.")
    else:
        print("Member deleted!")

def add_trainer():
    name = read_text("Enter trainer name: ", 50)
    specialization = read_text("Enter specialization: ", 50)

    cur.execute("""
        INSERT INTO trainers (trainer_name, specialization)
        VALUES (%s, %s)
    """, (name, specialization))

```

```

con.commit()
print("Trainer added! Trainer ID:", cur.lastrowid)

def view_trainers():
    cur.execute("""
        SELECT trainer_id, trainer_name, specialization
        FROM trainers
        """)

    rows = cur.fetchall()

    if len(rows) == 0:
        print("No trainers found.")
    else:
        for row in rows:
            print("\n-----")
            print("Trainer ID:", row[0])
            print("Name:", row[1])
            print("Specialization:", row[2])

def reports():
    cur.execute("SELECT COUNT(*) FROM members")
    print("Total members:", cur.fetchone()[0])

    cur.execute("SELECT COUNT(*) FROM trainers")
    print("Total trainers:", cur.fetchone()[0])

    cur.execute("SELECT AVG(age) FROM members")
    average = cur.fetchone()[0]

    if average is None:
        print("Average age: No age data available")
    else:
        print("Average age:", round(average, 1))

while True:
    print("\n--- Gym Management System ---")
    print("1. Add member")
    print("2. View members")
    print("3. Search member")
    print("4. Update member phone")
    print("5. Delete member")

```

```

print("6. Add trainer")
print("7. View trainers")
print("8. Reports")
print("9. Exit")

choice = input("Enter your choice: ").strip()

try:
    if choice == "1":
        add_member()
    elif choice == "2":
        view_members()
    elif choice == "3":
        search_member()
    elif choice == "4":
        update_member()
    elif choice == "5":
        delete_member()
    elif choice == "6":
        add_trainer()
    elif choice == "7":
        view_trainers()
    elif choice == "8":
        reports()
    elif choice == "9":
        break
    else:
        print("Invalid choice.")

except mysql.connector.Error as error:
    con.rollback()
    print("Database error:", error)

cur.close()
con.close()

print("Goodbye!")

```

OUTPUT SCREENS

1. Main Menu & Registering a Member

```
--- Gym Management System ---
1. Add member
2. View members
3. Search member
4. Update member phone
5. Delete member
6. Add trainer
7. View trainers
8. Reports
9. Exit
Enter your choice: 1
Enter name: Jyotishman Chetia
Enter age: 18
Enter 10-digit phone number: 9395578082
Enter trainer ID (optional, press Enter to skip):
Member added! Member ID: 4
```

2. Viewing Members

```
Enter your choice: 2

-----
Member ID: 4
Name: Jyotishman Chetia
Age: 18
Phone: 9395578082
Trainer: Not assigned
```

OUTPUT SCREENS (Cont.)

3. Adding a Trainer

```
Enter your choice: 6
Enter trainer name: Rakesh Kumar
Enter specialization: Weightlifting
Trainer added! Trainer ID: 1
```

4. Reports

```
Enter your choice: 8
Total members: 1
Total trainers: 1
Average age: 18.0
```

SHORTCOMINGS / LIMITATIONS

1. The project runs through a simple text-based menu in the terminal and does not include a graphical user interface (GUI), which may limit user-friendliness for non-technical users.
2. There is no login or authentication system for the administrator, so anyone who opens the program can modify trainers or members.
3. Member verification depends only on the unique Phone Number entered by the user, and there is no additional method (like biometrics) to confirm identity at the gym.
4. Data stored in the database is not encrypted, which would be required for high-security applications handling sensitive personal data.
5. The system is designed for small-scale gyms and may not provide advanced reporting features like revenue graphs, attendance charts, or membership expiry alerts.

REFERENCES (BIBLIOGRAPHY)

The following books, online documentation, resources, and tools were used to design and complete this project:

1. Python Official Documentation

For understanding Python syntax, functions, modules, and exception handling.

<https://docs.python.org/>

2. MySQL Official Documentation

Used to understand SQL commands, database creation, table constraints, primary and foreign keys, and transactions.

<https://dev.mysql.com/doc/>

3. mysql-connector-python Library

Helpful for learning how Python connects with MySQL, executes queries, commits transactions, and handles SQL errors.

<https://pypi.org/project/mysql-connector-python/>

4. CBSE Computer Science (083) Curriculum 2025-26

For guidelines on project structure, database connectivity, and the required format for project work.

THANK YOU
